Bachelor's Thesis

Implementing Floating-Point Support in a Nanopass-Python Compiler

Marius Spill

Examiner: Prof. Dr. Peter Thiemann
Advisers: TO DO

Albert-Ludwigs-University Freiburg
Faculty of Engineering
Department of Computer Science
Chair for Programming Languages

September 15, 2026

Writing period

06. 15. 2026 – 09. 15. 2026

Examiner

Prof. Dr. Peter Thiemann

Advisers

TODO

Declaration

I hereby declare that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.
I hereby also declare that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date	Signature

Abstract

foo bar

Contents

1 Introduction

1.1 Subchapter 1

The Hook: Why do we care about small/incremental compiler frameworks (like Nanopass)?

1.2 Subchapter 2

The Problem: What happens when a framework or downstream intermediate representation (IR) lacks native floating-point types? (e.g., restriction to purely integer arithmetic limits the computational expressiveness of the language). The Contribution: A brief high-level summary of what you added to the pipeline.

2 Background

The Nanopass Framework: Explain how the host compiler operates (small, discrete passes transforming highly specific AST micro-languages). Floating-Point Representation: Briefly introduce the target representation semantics (e.g., IEEE 754 standards) and how the target machine or runtime expects floats to be structured compared to integers.

3 Design Architecture

3.1 Design Philosophy

The governing principle of this extension was functionality first, optimisation second. This drove two central decisions. First, floats are represented as 64-bit doubles rather than 32-bit single-precision values, in order to preserve fidelity with Python's native semantics. Second, floats are stored as boxed heap objects rather than as raw values, accepting the cost of indirection in exchange for reusing the compiler's existing tuple and garbage-collection machinery. Performance optimisations were deferred until the base functionality for floats was complete, and were only revisited where time allowed.

A related decision concerns mixed-type arithmetic. In any operation mixing an integer and a float, the integer operand is implicitly promoted to a float before the operation is performed, matching Python's native behaviour. This keeps a single arithmetic path for floats and confines the type difference to a conversion step at the operands. The design and placement of this conversion are discussed in Section 3.3.

3.2 Float Representation on the Heap

The garbage collector uses bit tagging to distinguish pointers, which it must follow, from raw values, which it must leave untouched. Integers are stored shifted one bit to the left, so that their least-significant bit (LSB) is 0, marking

the word as a value the collector should not trace. Heap pointers instead carry an LSB of 1, signalling the collector to follow them. Every structure stored on the heap (tuples, closures, and so on) is laid out as a block of words prefixed by a header word.

A double in the IEEE-754 format cannot be tagged in this way, because all 64 bits are needed to encode the value, leaving no bit free for a tag. Worse, the bit that serves as the tag for integers corresponds, in a double, to part of the mantissa; repurposing it would distort the value entirely.

Floats must nonetheless be storable on the heap, since the language permits them as fields of objects, as function arguments, and as elements of tuples. Restricting the language to avoid this was rejected on principle, as the guiding philosophy stated above places functionality over performance. A representation compatible with the existing tagged collector was therefore required.

3.2.1 Alternatives

One option would have been to store floats as raw values directly on the heap. This would require overhauling the garbage collector to move away from bit tagging towards a layout map, that is, a description of the heap that records, for each structure, which words are pointers and which are raw values. Such a scheme would offer a scalable and memory-efficient way to represent any value directly on the heap, and would allow values to be stored inside any other structure without indirection. The drawback is that it conflicts with the architecture of the compiler as built so far, and that constructing and maintaining layout maps would introduce considerable overhead.

The approach I chose instead was to box every float inside a two-word heap block: one header word and one word holding the value of the float. The header identifies the block as a boxed float. This yields a uniform representation that can be used for literals, for variables, and for floats nested

in any heap structure, all through the same interface.

The intended behaviour is for the collector to recognise a boxed float from its header and to copy the payload word verbatim, without interpreting those raw bits as a pointer. As discussed in Section 3.3.2, this header-aware handling is not yet in place.

The advantage of boxing is that it offers a uniform design that delivers full functionality with little overhead and fits almost seamlessly into the existing compiler. The disadvantage is the inefficiency that arises from having to reach inside the box on every access to the value, and from allocating a fresh box for every float that is produced, which increases pressure on the collector. These costs were judged acceptable, since the trade-off aligned with the design philosophy of providing functionality before performance.

3.3 Conversion of Integers to Floats in Coercion Expressions

Python implicitly promotes integers to floats in mixed-type operations—`3 + 3.14` evaluates to `6.14`, not a type error. Supporting this behaviour follows directly from the design philosophy of preserving Python's native semantics.

The detection of cross-type operations is performed in the heap allocation pass. This pass already identifies floats to box them, and more generally decides what must happen to each piece of data—whether it needs boxing, unboxing, or conversion. Placing the coercion logic here keeps these semantic decisions in one place.

When the pass detects an operation in which at least one operand is a float and the other is an integer, it wraps the integer operand in a new unary operation, `int_to_float`. Structurally, this operation behaves just like any other unary expression (such as negation); it flows through the subsequent passes unchanged until it reaches instruction selection, where it is lowered to

the RISC-V `fcvt.d.l` instruction that converts a 64-bit integer value to a double-precision float.

3.3.1 Alternatives

One alternative would have been to introduce a dedicated coercion pass before the heap allocation pass. This would separate concerns more cleanly, since one could argue that the purpose of the allocation pass is heap allocation, not type coercion. While technically correct, the trade-off would be introducing an entire new pass for a single transformation in one specific case. This option was rejected as disproportionate; however, should the language require additional type conversions in the future, a unified coercion pass would become architecturally justified.

A second and simpler alternative would have been to handle coercion entirely in instruction selection, using the type dictionary to detect mixed-type operands and emitting the conversion inline. This would confine the change to a single file, making it the least complex solution. The drawback is that instruction selection would take on semantic work—deciding *what* conversion is needed—on top of its existing role of deciding *how* to lower operations to machine instructions. The chosen approach keeps these responsibilities separate: the allocation pass decides that a conversion is needed, and instruction selection decides how to implement it. This separation also makes a future migration to a dedicated coercion pass straightforward, since the conversion is already represented as an explicit operation in the intermediate representation.

3.3.2 Possible Improvements

Efficiency could be improved by replacing bit tagging with a full layout or type map, which would permit storing raw floats directly on the heap rather than boxing them. Complementarily, the register allocator could be extended

to distinguish floats from integers and to maintain a separate region of the stack for floating-point values, so that raw floats could be spilled to the stack just like integers, instead of being boxed and sent to the heap.

Independently of performance, the boxing scheme is not yet fully sound under the copying collector. After an object is copied, the collector scans each of its words and tests the LSB to decide whether the word is a pointer to follow. Because a boxed float's payload is a raw 64-bit double, a value whose low bit happens to be 1 would be misinterpreted as a tagged pointer and erroneously followed. Making boxing correct therefore requires the header-aware traversal described above, so that the payload word of a float box is copied verbatim and never scanned as a pointer.

4 Implementation

The integration of floating-point operations requires a full-vertical extension across the compiler pipeline. This chapter details the type systems, lowering mechanisms, and code generation required to support 64-bit double-precision floats within the RV64GC architecture.

4.1 Interpreter Extension and Phase-Driven Design

Before implementing the concrete compiler backend, the language's dynamic semantics were established by extending the reference interpreter. Because the interpreter defines the ground-truth behavioral specification of the language, this execution layer must be finalized first to ensure semantic parity with the compiler's eventual target output.

To establish clear verification metrics, a differential test suite containing half a dozen Python programs was constructed. These test cases duplicated existing integer-based control flow, arithmetic expressions, and native built-in functions (such as `print()`), substituting integer literals with floating-point values. The immediate implementation goal was to allow the interpreter to execute this test suite natively without throwing exceptions, matching Python's runtime behavior exactly.

Extending the interpreter required systematic modifications across three core subsystems of the pipeline: the abstract syntax tree (AST) parser, the static type-checker, and the evaluation semantics engine.

4.1.1 Syntactic Extensions (The Parser)

Extending the frontend parser was highly deterministic because the host language's `ast` module natively recognizes floating-point tokens. The internal AST grammar definition was upgraded to allow literal expression nodes (`EConst`) to hold float primitives alongside integers. Additionally, the type-mapping routines were extended to recognize and propagate explicit variable type annotations, allowing patterns such as `a : float = 3.14` to pass successfully through the front-end token stream.

4.1.2 Static Semantics and the 64-Bit Precision Decision

The primary design challenge emerged during the static analysis phase within the type-checking subsystem. Because the compiler targets the RV64GC architecture—which natively supports both single-precision ('F') and double-precision ('D') floating-point extensions—implementing 32-bit floats offered tempting advantages in implementation expediency, specifically minimizing heap pressure and reducing the footprint of boxed values. However, forcing a 32-bit representation would introduce semantic drift from standard Python behavior, which treats all floating-point numbers as 64-bit doubles; common decimals like `0.1` or `3.14` would suffer silent precision loss. This directly conflicts with the design philosophy established in Chapter 3, which prioritises preserving Python's semantics over efficiency and implementation simplicity.

A hybrid architecture—selectively flagging exact base-2 powers (e.g., $0.5 = 2^{-1}$) as 32-bit floats and remaining decimals as 64-bit doubles—was also explored and rejected for the same reason: Python does not expose mixed-precision literals, and forcing such a distinction would create an un-Pythonic dialect while introducing significant complexity into the code generator.

To preserve strict semantic fidelity, I opted for a unified 64-bit double-precision model targeting the RISC-V 'D' extension, accepting the trade-off of higher memory overhead. Consequently, the type validator enforces a

single constraint: verifying that literals do not exceed the 64-bit IEEE 754 infinity threshold.

4.1.3 Type-System Generalization

Beyond validating literals, the type-checker's expression evaluation routines needed a clean refactoring. Originally, the compiler assumed almost everything was an integer, evaluating binary operators and functions like `print()` by checking them directly against a hardcoded `TInt()` type. To support floating-point numbers without copying and pasting the same validation logic everywhere, this rigid approach was replaced with a more flexible system. A new helper function, `check_types_supported`, was introduced to validate expressions against a list of acceptable types instead of just one. For this first iteration, operands must still match exactly—meaning mixed-type operations like `float + int` are blocked for now. However, this refactor ensures that expressions evaluate to and return their correct type primitive (like `TFloat()`) rather than defaulting to an integer, laying the clean groundwork for future cross-type arithmetic.

4.1.4 Interpreter Semantics

Extending the runtime behavior in the interpreter was the most straightforward step because the interpreter itself is written in Python, which already supports 64-bit floating-point math natively.

In `semantics.py`, the evaluation rules for binary expressions were simply updated to accept float values alongside integers. When the interpreter encounters an operation like addition, it handles the AST nodes by passing the raw values directly to Python's native operators. By leveraging the host language's execution environment this way, the interpreter was immediately able to run the floating-point test suite with perfect accuracy.

4.2 Heap Allocation for Floats

The decision to box every float carried two practical consequences for the allocation pass. From this pass onwards, a float is represented as a pointer to its box, so any operation that consumes a float must first unbox it to recover the underlying value, and any operation that produces a float must box the result again before it can be stored back on the heap. The difficulty is that the allocation pass must know, for each operand, whether it is a float; otherwise it would apply the arithmetic to the box pointer rather than to the value it points to. The type information needed to make this decision was computed earlier, by the type checker, so the task was to make that information available at this point in the pipeline and to consult it for each operand.

The first step was a function that determines whether a given expression yields a float. Whenever it does, the uniform boxing design dictates that the operand must be unboxed before the operation and, where the operation itself yields a float, boxed again afterwards.

A complication arose for values held inside tuples (and for closures, functions, and objects, which are all represented as tuples at this stage). The type checker does verify the type of every element of a tuple, but that per-element information was not preserved beyond type checking, so by the time the allocation pass ran there was no way to recover whether the element at a given position was a float. The expression's shape alone is sufficient for literals and for nested arithmetic, but not for accesses into a tuple or for plain variables, where the type is not visible in the expression itself. To resolve this, I returned to the type checker and preserved the type information it had already derived.

4.2.1 A Dictionary to Preserve Type Information

The result was a dictionary that, for each such structure, uses its identifier as the key and stores its fields or elements together with their corresponding types as the value. This dictionary is returned from the type checker to the main compiler driver, stored there, and passed on to every pass that requires it.

Up to this point in the pipeline, two passes consume the dictionary. The first is uniquify, which renames variables and must therefore update the dictionary's keys so that the recorded types remain reachable under the new names. The second is the heap allocator, for which the dictionary was originally introduced, where it is used to decide whether a given element is a float.

4.2.2 Handling Floats inside the Heap Allocator

When one or more operands of an operation are floats, the allocator emits a block that first extracts each float value from its box, then performs the operation on the unboxed values, and finally, where the result is itself a float, stores it back into a freshly allocated box. When none of the operands are floats, the operation is emitted unchanged, exactly as before.

4.3 Float Literal Support End to End

With the heap-based boxing design fully implemented, the next task was to support float literals through the entire compiler pipeline, from instruction selection down to executable assembly. The test program `float_literal.py` (see Appendix) served as the target: it assigns a float literal to a variable and prints it.

4.3.1 Instruction Selection

The first pass that required modifications was instruction selection. As a prerequisite, I extended the register library to include all 32 floating-point registers defined by RISC-V (see Appendix): the temporary registers `ft0`–`ft11`, the saved registers `fs0`–`fs11`, and the argument registers `fa0`–`fa7`. Under the boxed-float strategy most of these registers are not yet used, since every float value resides on the heap and is only loaded into a register for a single operation before being stored back. Nonetheless, defining the full register set lays the groundwork for future optimisations such as keeping floats in registers across operations or spilling to the stack instead of the heap.

Inside the pass itself, I threaded the types dictionary from the compiler driver down to instruction selection so that the pass could determine whether a value should be directed to a floating-point register or an integer register. Accessing a boxed float requires unboxing: loading from byte offset 7 of the tagged heap pointer to reach the payload field of the box.

4.3.2 Patch Instructions for Literals

The final pass that required changes was patch instructions, which lowers abstract move operations into concrete RISC-V assembly. To support this, I first extended the RISC-V AST with a new instruction node for floating-point moves. RISC-V provides three variants: `fmv.d.x` (integer register to float register), `fmv.x.d` (float register to integer register), and `fmv.d` (float register to float register). I defined all three in the AST.

Inside the pass, when a move instruction is encountered, I introduced a four-way case distinction based on whether the source and destination are floating-point or integer registers. Each combination selects the appropriate move instruction: `fmv.d` when both are float registers, `fmv.d.x` when only the destination is a float register, `fmv.x.d` when only the source is a float

register, and the standard integer `add` (used as a move) when neither is.

4.3.3 Extension of the Runtime

To support printing floats, I added a `print_float` function to the C runtime (`runtime.c`). This function receives a double-precision value in `fa0`—following the RISC-V calling convention for floating-point arguments—and prints it using `printf`. This mirrors the existing `print_int64` function that the base compiler uses for integers.

4.3.4 Walkthrough of the Generated Assembly

To illustrate the end-to-end mechanics, consider how the compiler handles the statement `a: float = 3.14`. The float value is converted to its IEEE 754 bit representation as a 64-bit integer and loaded into an integer register using `li`. It is then stored at byte offset 7 from the tagged heap pointer, which places it in the payload field of the box:

```
li      a2, 4614253070214989087
add     t0, zero, a2
sd      t0, 7(a1)
```

When the program calls `print(a)`, the compiler emits instructions to unbox the float: it loads the raw bits from the heap back into an integer register, then uses `fmv.d.x` to reinterpret those bits as a double-precision value in the floating-point register `fa0`, and finally calls the runtime's `print_float`:

```
ld      a1, 7(a1)
fmv.d.x fa0, a1
call    print_float
```

4.4 Arithmetic for Floating-Point Values

With literals and printing working, the next step was arithmetic. The base compiler only supported addition and subtraction for integers, so multiplication (`*`) and division (`/`) had to be added to the parser, type checker, and every intermediate AST regardless of floats. For floating-point values, RISC-V provides native double-precision instructions for all four operations: `fadd.d`, `fsub.d`, `fmul.d`, and `fdiv.d`.

4.4.1 Transient Use of Float Registers

The central design choice for arithmetic was to keep float registers as a transient detour rather than a place where values live. The instruction selection pass emits the following sequence for every float operation: move both unboxed operands from integer registers into `fa0` and `fa1` with `fmv.d.x`, perform the operation (e.g. `fadd.d fa0, fa0, fa1`), move the result back to an integer register with `fmv.x.d`, and box it on the heap immediately. Float registers are therefore never live across instruction boundaries from the register allocator's perspective.

This approach required two supporting changes. First, the heap allocation pass already introduces temporary variables for each intermediate result of a nested expression. These temporaries must be recorded in the type dictionary as `TFloat` so that downstream passes can identify them. Second, the instruction selection pass must distinguish float operations from integer ones by checking the operand types. However, the monadic normalisation pass—which flattens nested expressions into a sequence of assignments to fresh temporaries—originally did not have access to the type dictionary, leaving its temporaries untyped. To solve this, I threaded the type dictionary through the monadic pass. When the pass creates a new temporary for a non-atomic expression such as `ETupleAccess(EVar(v), 0)` where `v` is typed

as `TFloat`, the temporary inherits that type. This ensures that operand types are available in instruction selection uniformly for both arithmetic and comparisons.

4.4.2 Registering Float Registers in the Allocator

Because `fa0` and `fa1` now appear explicitly in the instruction stream, the register allocator must know about them: any register that shows up in the interference graph needs a colour mapping, otherwise the graph colouring algorithm crashes when it encounters an unknown register as a neighbour of a variable. I added both registers to the blacklist at the beginning of `REG_ORDER`, which assigns them a negative colour. This ensures the allocator recognises them but never assigns a variable to them—which is exactly the desired behaviour, since float values are boxed to the heap immediately and never need to be spilled to a float register.

4.4.3 Walkthrough of the Generated Assembly

The generated code for a float addition such as `a + b` mirrors the unbox–operate–box pattern. Both operands are loaded from the heap, moved into float registers, added, and the result is stored back:

```
ld      a3, 7(a3)
fmv.d.x fa0, a2
add     a2, zero, a3
fmv.d.x fa1, a2
fadd.d  fa0, fa0, fa1
fmv.x.d a2, fa0
add     t0, zero, a2
sd      t0, 7(a1)
```

The first two lines unbox the first operand: `ld` loads the raw bits from the heap (at tagged offset 7), and `fmv.d.x` reinterprets them as a double in

`fa0`. Lines three and four do the same for the second operand into `fa1`. The `fadd.d` performs the addition. Finally, `fmv.x.d` moves the result back to an integer register, and `sd` stores it into a freshly allocated box on the heap.

4.5 Comparing Operations for Floats

With arithmetic in place, the next step was comparisons. The RISC-V AST had to be extended with three native double-precision comparison instructions: `feq.d`, `flt.d`, and `fle.d`. As with integers, comparisons appear in two contexts that must be handled differently: assigning the result of a comparison to a variable, and branching on a comparison (e.g. `if`/`else`).

4.5.1 Assigning Comparison Results

When a comparison result is assigned to a variable, the compiler loads both operands, checks their types, and—if they are floats—moves them into the designated float registers to perform the comparison. The result is a 0 or 1 in an integer register, which is then tagged like any other integer and stored in the destination variable.

RISC-V only provides `feq.d` (equal), `flt.d` (less than), and `fle.d` (less or equal). The missing operators are derived with simple transformations: greater-than and greater-or-equal swap the operands (`a > b` becomes `flt.d rd, fb, fa`), while not-equal uses `feq.d` followed by an `xor` to invert the result.

4.5.2 Branching on Comparison Results

The branching case differs structurally from the integer path. For integers, RISC-V can compare and branch in a single instruction (e.g. `blt rs1, rs2, label`). For floats, this is not possible—no float branch instructions exist. Instead, the compiler emits the float comparison into a temporary integer

register and then branches on that register with `bne` (branch if not equal to zero).

4.5.3 Walkthrough of Generated Assembly

The following walkthrough shows the branching case. As with any float operation, both values are unboxed from the heap and moved into float registers. The `flt.d` instruction compares them and writes 1 or 0 to an integer register. The result is then tagged and the branch is taken accordingly:

```
ld      a2, 7(a2)
ld      a1, 7(a1)
fmv.d.x fa0, a2
fmv.d.x fa1, a1
flt.d   a1, fa1, fa0
slli    a1, a1, 1
li      t1, 0
bne     a1, t1, body_6
j       orelse_7
```

4.6 Type Coercion for Mixed Integer and Float Operations

The first change required was relaxing the type checker. Previously, binary operations required both operands to have the same type. This constraint was loosened so that operands need only be individually valid for the operation—for instance, both `TInt` and `TFloat` are permitted for arithmetic—without requiring them to match. When at least one operand is a float, the result type is `TFloat`.

With the type checker permitting mixed-type expressions, the heap allocation pass handles the actual coercion. This pass already inspects each operand to determine whether it is a float and requires unboxing. When one

operand is a float and the other is an integer, the pass wraps the integer in the `int_to_float` unary operation introduced in Section 3.3. The wrapped expression flows through the monadic and explicate passes without special handling.

In instruction selection, the `int_to_float` operation is lowered to a short sequence: the integer is loaded into a temporary register, untagged by shifting right by one bit (since the compiler stores integers in tagged form), and then converted to a double-precision float using the RISC-V `fcvt.d.l` instruction. The resulting float value is moved back to an integer register, ready for use in the subsequent arithmetic operation.

4.6.1 Walkthrough of Generated Assembly

The following listing shows the generated code for `3.2 * 3`, where the first operand is a float and the second is an integer. The float is unboxed from the heap as usual, while the integer literal 3 is loaded in its tagged form (value 6), untagged, and converted to a double before the multiplication:

```
ld      a3, 7(a3)
li      a2, 6
srli    a2, a2, 1
fcvt.d.l fa0, a2
fmv.x.d a2, fa0
fmv.d.x fa0, a3
fmv.d.x fa1, a2
fmul.d  fa0, fa0, fa1
fmv.x.d a2, fa0
```

The first line unboxes the float operand. Lines two through five handle the integer: `li` loads the tagged value 6, `srli` untags it to 3, and `fcvt.d.l` converts the integer 3 to the double 3.0 in `fa0`. The result is then moved back to an integer register with `fmv.x.d`. Finally, both operands are moved into float registers and the multiplication is performed.

5 Evaluation

Correctness Validation: Instead of standard benchmarks or accuracy percentages, compiler evaluations validate determinism and compliance. Explain your test suite framework. Test Cases: Discuss how you validated corner cases (e.g., parsing precision limits, type coercion, complex floating-point expressions). Performance (Optional): If you optimize passes or look at compilation times, present it here. Otherwise, focus heavily on functional correctness and compilation validation.

6 Conclusion

Short summary of your execution and a couple of bullet points on future work (e.g., supporting double-precision, floating-point optimization passes, or array allocations of floating-point numbers).

Bibliography